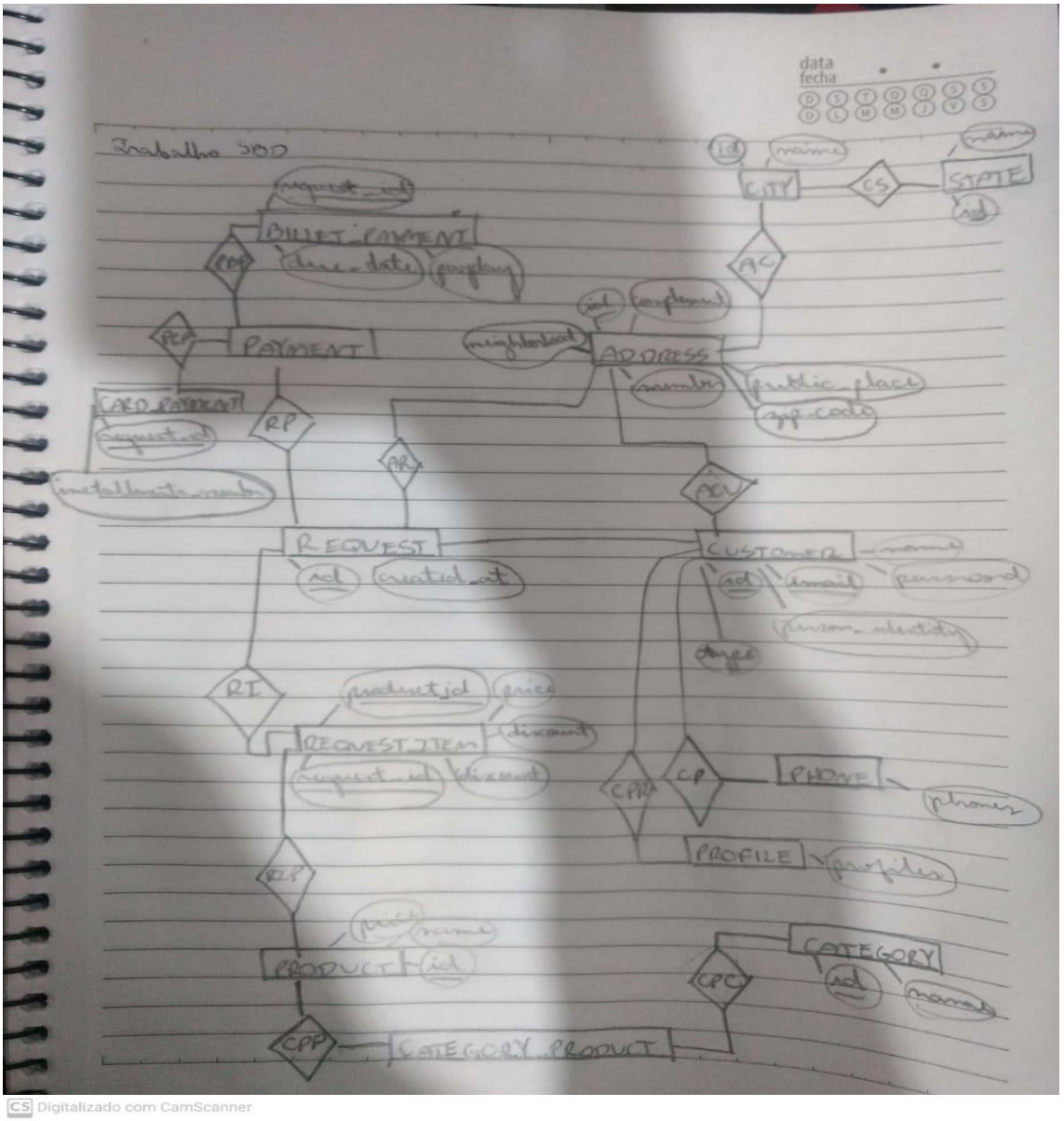
Trabalho Prático de SBD

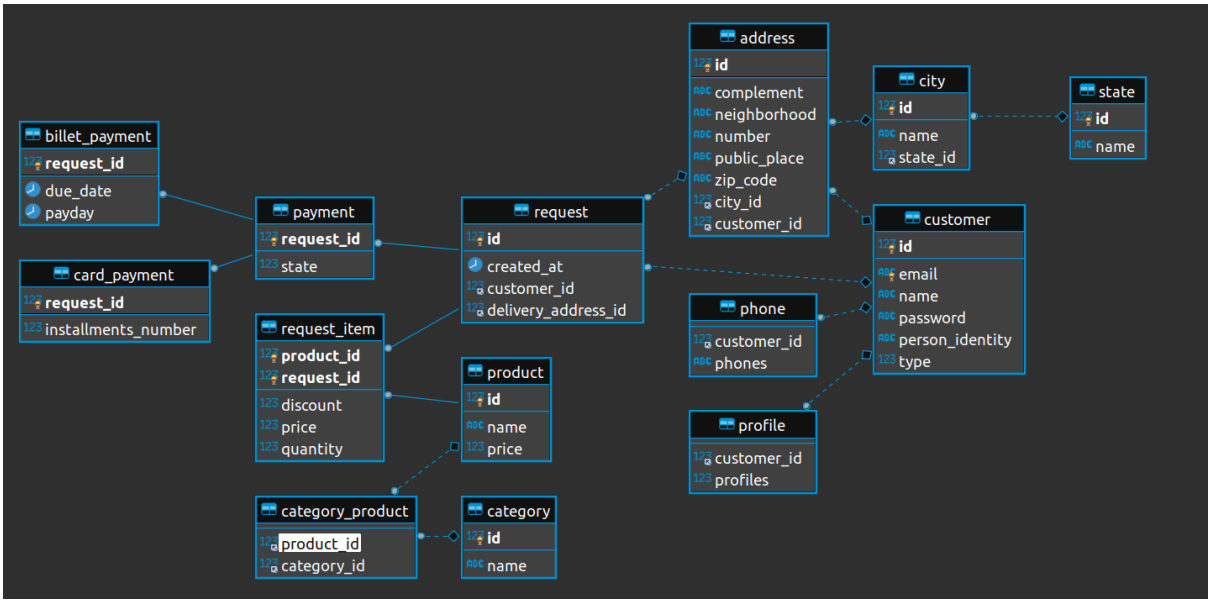
Matheus José da Costa 11711BCC008
Leonardo Essia 11712BCC009

a) A aplicação escolhida é a simulação de um e-commerce. Em suma, um cliente se cadastra no site, com email, nome e senha e o tipo de pessoa (pessoa física ou pessoa jurídica), e este tem acesso a produtos (nome e preço), esses que são divididos em categorias (nome da categoria). O cliente, ao requisitar um produto (adicionar na sacola) faz o pagamento no Cartão de Crédito (escolhendo o número de parcelas) ou no Boleto (data de pagamento e dia que o pagamento foi efetivado). E além disso o cliente ao escolher algum produto, preenche os dados para envio, como Endereço (nome da cidade, nome do estado, CEP, referência), além do seu número de telefone para contato.

b)



c) Modelo implementado no SGBD MySQL:




```
create table category
(
   id int auto_increment
       primary key,
   name varchar(255) null
);

create table customer
(
   id int auto_increment
       primary key,
   email varchar(255) null,
   name varchar(255) null,
   password varchar(255) null,
   person_identity varchar(255) null,
   type int null,
   constraint UK_dwk6cx0afu8bs9o4t536v1j5v
       unique (email)
);

create table phone
(
   customer_id int not null,
   phones varchar(255) null,
   constraint FKgvgc4f2c3qkiswyv6bbs0br1p
```

```
        foreign key (customer_id) references customer (id)
);

create table product
(
    id int auto_increment
        primary key,
    name varchar(255) null,
    price double null
);

create table category_product
(
    product_id int not null,
    category_id int not null,
    constraint FKfr6rjc04htbtc3xas2b9xmq7r
        foreign key (category_id) references category (id),
    constraint FKssroqj2vyiaujfleklq1ifigj
        foreign key (product_id) references product (id)
);

create table profile
(
    customer_id int not null,
    profiles int null,
    constraint FK88rruc7qawl75mscnnwrah9sc
        foreign key (customer_id) references customer (id)
);

create table state
(
    id int auto_increment
        primary key,
    name varchar(255) null
);

create table city
(
    id int auto_increment
        primary key,
    name varchar(255) null,
    state_id int null,
    constraint FK6p2u50v8fg2y0js6djc6xanit
```

```
        foreign key (state_id) references state (id)
);

create table address
(
    id int auto_increment
        primary key,
    complement varchar(255) null,
    neighborhood varchar(255) null,
    number varchar(255) null,
    public_place varchar(255) null,
    zip_code varchar(255) null,
    city_id int null,
    customer_id int null,
    constraint FK93c3js0e22ll1xlu21nvrhqgg
        foreign key (customer_id) references customer (id),
    constraint FKpo044ng5x4gynb291cv24vtea
        foreign key (city_id) references city (id)
);

create table request
(
    id int auto_increment
        primary key,
    created_at datetime null,
    customer_id int null,
    delivery_address_id int null,
    constraint FK6wuyy6femh1tl1jxmw1ilrs6b
        foreign key (customer_id) references customer (id),
    constraint FKn579kjmk2jkthnj21hyv20h54
        foreign key (delivery_address_id) references address (id)
);

create table payment
(
    request_id int not null
        primary key,
    state int null,
    constraint FK5pgyup82pk40rm2mo6jeqrbi7
        foreign key (request_id) references request (id)
);

create table billet_payment
```

```
(
   due_date datetime null,
   payday datetime null,
   request_id int not null
       primary key,
   constraint FKbk36yb9dp0cpr3beqn66y8xmb
       foreign key (request_id) references payment (request_id)
);

create table card_payment
(
   installments_number int null,
   request_id int not null
       primary key,
   constraint FK2k4ugt7m1xsxvb15xt9y8dnr2
       foreign key (request_id) references payment (request_id)
);

create table request_item
(
   discount double null,
   price double null,
   quantity int null,
   product_id int not null,
   request_id int not null,
   primary key (product_id, request_id),
   constraint FKd2t6dw5gx34u1me7451b64xq3
       foreign key (request_id) references request (id),
   constraint FKrtyfreo7es5rpkasqslrpvm5v
       foreign key (product_id) references product (id)
);
```

d)

data
fecha

d) (1) Usando Expressões

* Listar nome de todos clientes e a rua em que mora, bairro e complemento

Π customer.name, address.neighborhood, address.public_place, address.complement (CUSTOMER $\bowtie_{CUSTOMER.ID = ADDRESS.ID}$ ADDRESS)

* Listar todos os pedidos (id do pedido), data de criação, o nome e o cpf de quem fez o pedido

Π CUSTOMER.name, CUSTOMER.PERSON_IDENTITY, REQUEST.ID, REQUEST.CREATED_AT (CUSTOMER $\bowtie_{CUSTOMER.ID = REQUEST.CUSTOMER_ID}$ REQUEST)

* Listar os telefones do cliente, com seu nome e cpf:

Π CUSTOMER.NAME, CUSTOMER.PERSON_IDENTITY, PHONE.PHONES (CUSTOMER $\bowtie_{CUSTOMER.ID = PHONE.CUSTOMER_ID}$ PHONE)

Digitalizado com CamScanner

data
fecha

② Usando sequência de operação com o resultado final atribuído a var. R.

* Listar nome de todos clientes e seu endereço completo

R1 ← CUSTOMER ⋈ CUSTOMER.ID=ADDRESS.ID ADDRESS

R ← Π CUSTOMER.NAME, ADDRESS.NEIGHBORHOOD, ADDRESS.PUBLIC_PLACE, ADDRESS.COMPLEMENT (R1)

* Listar todos os pedidos (id do pedido), data de criação, o nome e o cpf de quem fez o pedido

R1 ← CUSTOMER ⋈ CUSTOMER.ID = REQUEST.CUSTOMER_ID REQUEST

R ← Π CUSTOMER.NAME, CUSTOMER.PERSON_IDENTITY, REQUEST.ID, REQUEST.CREATED_AT (R1)

* Listar os telefones dos clientes, com seu nome e cpf.

R1 ← CUSTOMER ⋈ CUSTOMER.ID = PHONE.CUSTOMER_ID PHONE

R ← Π CUSTOMER.NAME, CUSTOMER.PERSON_IDENTITY, PHONE.PHONES (R1)

CS Digitalizado com CamScanner

Essas expressões foram criadas a partir das seguintes consultas SQL:

```
select c.name, a.neighborhood, a.public_place, a.complement from trabalho_sbd.customer
as c
inner join trabalho_sbd.address as a
on c.id = a.id
```

```
select c.name, c.person_identity, r.id, r.created_at from trabalho_sbd.customer as c
inner join trabalho_sbd.request as r
on c.id = r.customer_id
```

```
select c.name, c.person_identity, p.phones from trabalho_sbd.customer as c
inner join trabalho_sbd.phone as p
on c.id = p.customer_id
```